TASK-5

PRIM'S ALGORITHM

Problem : AIM:
Given a weighted undirected graph. Find the sum of weights of edges of a MST.

Input : Given 2 integers N and M. N represents the number of vertices in the graph. M represents the number of edges b/w any 2 vertices.

Output : Print the summation of edges weights in the MST.

ALGORITHM STEPS:

1. Maintain two disjoint sets of vertices.
2. Select the cheapest vertex that is connected to the growing spanning tree & is not in the growing spanning tree & add it into the growing spanning tree.
3. Insert the vertices, that are connected to growing spanning tree, into the priority Queue.
4. check for cycles.

CODING :

```
# include < stdio.h >
# include < stdlib.h >
# include < limits.h >
# include < stdbool.h >
# define MAX_N 10001
# define INFINT_MAX
// structure for adjacency list node
struct AdjList Node {
    int dest;
    int weight;
    struct AdjlistNode* next;
};
```

```
// Structure for adjacency list
struct AdjList {
    Struct AdjListNode* head;
    };
    // Structure for graph
        Struct Graph {
            int v;
        Struct AdjList* array;
        };
    // Structure for min heap node
    Struct MinHeapNode {
            int v;
            int key;
        };
// Structure for min heap
    Struct MinHeap {
        int size;
        int capacity;
            int* pos;
    Struct MinHeapNode** array;
    };
// Create a new adjacency list node
Struct AdjListNode* newAdjListNode(int dest, int weight) {
    Struct AdjListNode* newNode = (struct AdjListNode*) malloc
                                    (sizeof(struct AdjListNode));
    newNode->dest = dest;
    newNode->weight = weight;
    newNode->next = NULL;
    return newNode;
    }
    // Create a graph with V vertices
    Struct Graph* createGraph(int V) {
```

```
Struct Graph* graph = (struct Graph*)malloc(size of(struct Graph));
        graph->V = V;
        graph->array = (struct AdjList*)malloc(V*size of
                                        (struct AdjList));
        for (int i=0; i<V; ++i)
        graph->array[i].head = NULL;
            return graph;
        }

  // Add an edge to the graph
  void addEdge (struct Graph*graph, int src, int dest,
                                        int weight){
        // Add edge from src to dest
Struct AdjList Node* newNode = newAdjListNode(dest, weight);
   newNode->next = graph->array[src].head;
      graph->array [src].head = newNode;
   // Add edge from dest to src (undirected graph)
    newNode = new AdjList Node (src, weight);
    newNode -> next = graph->array [dest].head;
    graph-> array[dest].head = newNode;
     }
    // create a new min heap node
 Struct MinHeapNode* new MinNode (int V, int key){
    Struct MinHeapNode* minHeapNode = (struct
            MinHeapNode*)malloc(size of(struct MinHeap Node));
            minHeapNode -> V = V;
          minHeapNode -> key = key;
           return minHeapNode;
        }
   // create a min heap
Struct MinHeap* create MinHeap(int capacity){
      Struct MinHeap* minHeap = (struct MinHeap*)malloc (size of(
                                        struct Min Heap));
```

```
    minHeap->pos = (int*)malloc(capacity* sizeof(int));
      minHeap->size = 0;
  minHeap->capacity = capacity;
minHeap->array = (struct MinHeapNode**) malloc (capacity* size of
                                        (struct MinHeapNode*));
minHeap->pos = (int*) malloc (capacity* size of (int));
    minHeap->size = 0;
  minHeap->capacity = capacity;
  minHeap->array = (struct MinHeapNode**) malloc (capacity* size
                          of(struct MinHeapNode*));
        return minHeap;
    }
 // Swap two min heap nodes
void swapMinHeapNode (struct MinHeapNode** a, struct MinHeapNode
                                                  **b){
    struct MinHeapNode* t = *a;
            *a = *b;
            *b = t;
        }
// Heapify at given index
 void minHeapify (struct MinHeap* minHeap, int idx){
        int smallest = idx;
        int left = 2* idx + 1;
        int right = 2* idx+2;
   if (left < minHeap->size &&
          minHeap->array[left]->key < minHeap->array[smallest]->key)
          smallest = right;
   if (smallest != idx){
        // Update positions
    minHeap->pos[minHeap->array[smallest]->v] = idx;
minHeap->pos[minHeap->array[idx]->v] = smallest;
swapMinHeapNode(&minHeap->array[smallest], &minHeap->array[idx]);
    minHeapify(minHeap, smallest);
    }
  }
```

```
// Check if heap is empty
int isEmpty(struct MinHeap* minHeap) {
    return minHeap->size == 0;
}
// Extract minimum node from heap
struct MinHeapNode* extractMin(struct MinHeap* minHeap) {
    if (isEmpty(minHeap))
        return NULL;
struct MinHeapNode* root = minHeap->array[0];
    //Replace root with last node
struct MinHeapNode* lastNode = minHeap->array[minHeap->size-1];
  minHeap->array[0] = lastnode;
  // Update positions
  minHeap->pos[root->v] = minHeap->size-1;
  minHeap->pos[lastNode->v] = 0;
// Reduce heap size & heapify
--minHeap->size;
minHeapify(minHeap, 0);
    return root;
}
// Decrease key value of a vertex
void decreaseKey(struct MinHeap* minHeap, int v, int key) {
    int i = minHeap->pos[v];
    minHeap->array[i]->key = key;
while(i>0 && minHeap->array[i]->key < minHeap->array[(i-1)/2]->key):
  //swap this node with its parent
minHeap->pos[minHeap->array[i]->v] = (i-1)/2;
minHeap->pos[minHeap->array[(i-1)/2]->v] = i;
swapMinHeapNode(&minHeap->array[i], &minHeap->array[(i-1)/2]);
        i = (i-1)/2;
    }
}
```

```
    // Check if vertex is in min heap
int isInMinHeap(struct MinHeap* minHeap, int v){
    if (minHeap->pos[v]< minHeap->size)
      return 1;
      return 0;
  }
// Prim's algorithm to find MST weight
 int primMST (struct Graph* graph){
      int V = graph->V;
    int parent[V];
    int key[V];
  int totalWeight = 0;
struct MinHeap* minHeap = Create Min Heap(V);
   // intialize all keys as INF
 for (int v=0; v<V; ++v){
      parent [v] = -1;
       key [v] = INF;
 minHeap->array[v] = newMinHeapNode (v, key[v]);
 min Heap -> pos[v] = v;
}
// Make key of 0th vertex as 0 so that it's extracted first
    key [0] = 0;
minHeap->array[0]= newMinHeapNode (0, key[0]);
         minHeap->pos[0] = 0;
         minHeap->size = V;
 while (! isEmpty (minHeap)){
  struct MinHeapNode* minHeapNode = extractMin(minHeap);
      int u = minHeapNode->V;
 if (parent[u] != -1){
      // Add weight to total only if it's not the first vertex
    totalWeight += key[u];
  }
 struct AdjListNode* pCrawl = graph->array[u].head;
  while (pCrawl != NULL) {
    int V = pCrawl->dest;
```

```
    if (isInMinHeap(minHeap, v) && pCrawl -> weight < key[v]){
        key[v] = pCrawl->weight;
        Parent[v] = u;
        decreasekey(minHeap, v, key[v]);
    }
    pCrawl = pCrawl->next;
    }
    free(minHeap->pos);
    free(minHeap->array);
    free(minHeap);
return totalWeight;
}
    int main(){
    int N, M;
    Scanf("%d %d", &N, &M);
struct Graph* graph = CreateGraph(N);
    for (int i=0; i<M; i++) {
        int a, b, w;
    Scanf("%d %d %d", &a, &b, &w);
        // Convert to 0-based indexing
    addEdge(graph, a-1, b-1, w);
    }
    int result = primeMST(graph);
    Printf("%d\n", result);
    return 0;
}
```

VEL TECH - CSE	
EX NO	5
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	20
SIGN WITH DATE	14/10/24

Time complexity : $O((N+M)\log N)$ where N is vertices & M is edges

RESULT : For the given a weighted undirected graph. Finding the sum of weights of edges of a MST using prim's algorithm is executed & verified sucessfully.